DeFi on Solana · Week 2

Speaker script: every slide with its notes. Facts checked 11 to 14 Oct 2026; re-check anything marked "verify".

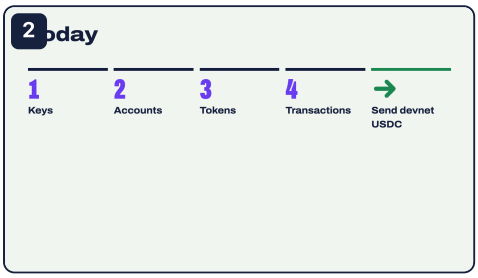


DeFi on Solana · Week 2

Week 2 goal: every mentee leaves with a devnet wallet, has sent a token, and can point at three things in an explorer: the fee, the token instruction, and the token account it created.

Everything in weeks 3 and 4 builds on one picture: wallet → token account → mint. If they leave with only one thing, make it that.

Faucets are the number one thing that breaks in class. Pre-fund a teacher wallet with devnet SOL and USDC before class.



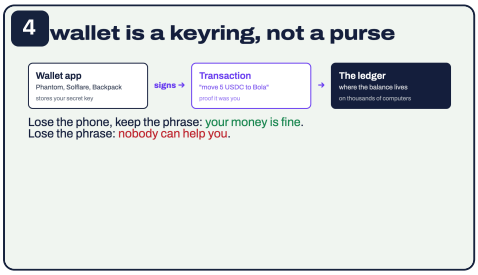
Today

No notes. Let the slide speak.



Keys

No notes. Let the slide speak.



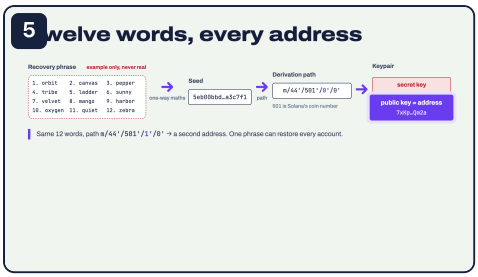
A wallet is a keyring, not a purse

The single most important mental model this week. Coins are not "in" the wallet app. They are entries on the ledger next to your address. The wallet app holds the key that lets you change those entries.

Consequences:

- If your phone breaks, install Phantom on a new phone, type your recovery phrase, and everything is there. Nothing was lost because nothing was on the phone.
- If someone gets your phrase, they can do the same thing on their phone. They don't need your device.

Compare a bank: your money is in the bank's ledger, and the bank decides who gets access. Here, the key decides.

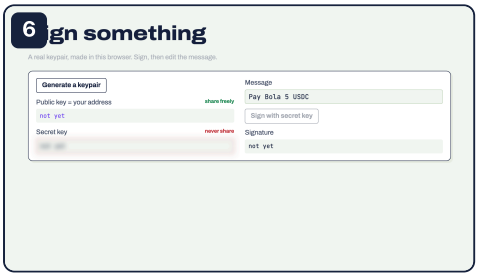


Twelve words, every address

Click through slowly.

1. The recovery phrase (also called seed phrase): 12 or 24 common words from a fixed list of 2,048 (the BIP-39 list). The example on screen is made up. Never show a real one.
2. The words are turned into a long number called the seed. This is one-way: you can't get the words back from the seed or anything after it.
3. A derivation path picks one keypair out of that seed. Solana wallets like Phantom use m/44'/501'/0'/0'. 501 is Solana's registered coin number.
4. Out comes a keypair: a secret key and a public key.
5. The public key, written in base58, is your address. That's what you share.
6. Change one number in the path and you get a completely different address from the same 12 words. That's how one phrase restores "Account 1", "Account 2" and so on in Phantom.

Your address is the same on mainnet, devnet and testnet. Balances are separate on each.



Sign something

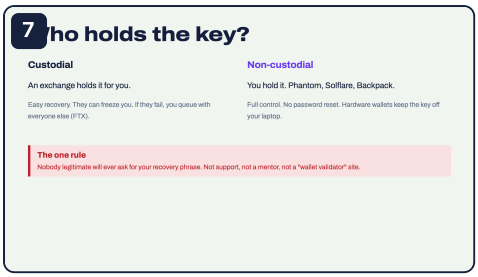
This is real Ed25519, the same signature scheme Solana uses. The key is generated in the browser and never leaves it.

Demo:

1. Generate. Point out the address looks like a Solana address (base58).
2. Click the blurred secret key to reveal it, then say: this is the one thing you never show anyone. Hide it again.
3. Sign "Pay Bola 5 USDC". Valid.
4. Change it to "Pay Bola 500 USDC" without re-signing. Invalid.

That is what a signature proves: this exact message was approved by whoever holds the secret key for this address. It does not prove who that person is in real life, and it doesn't prove the message is a good idea. A phishing site can get you to sign a perfectly valid transaction that drains you.

Signing a message (like "log in to this site") is different from signing a transaction (which moves funds). Wallets show which one you're doing. Read the popup.

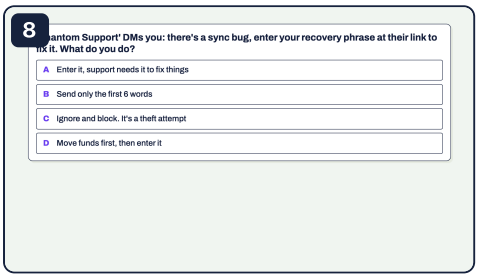


Who holds the key?

Wallet types to name quickly:

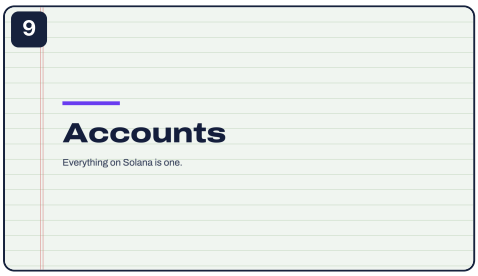
- Hot wallet: key on an internet-connected device (browser extension, phone app). Convenient, exposed.
- Hardware wallet (Ledger and others): the key stays on the device and signs there. Your laptop never sees it. Still vulnerable if you approve a bad transaction.
- Multisig: several keys must sign. Teams use this for treasuries and admin keys (Squads is the main one on Solana). Week 5's Drift hack shows a multisig can still be tricked.

Security checklist for mentees: write the phrase on paper, never in a photo or cloud note. Use a separate wallet for testing. Check the URL before connecting. Read every signing popup. Revoke or close wallets you don't use.



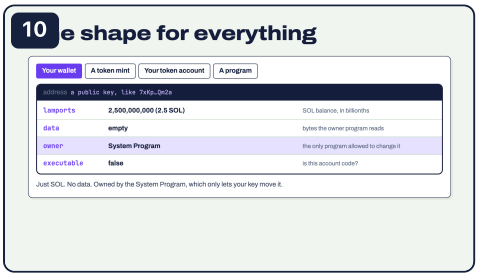
Quiz: 'Phantom Support' DMs you: there's a sync bug, enter your recovery phrase at their link to fix it. What do you do?

Option B is a real thing scammers ask for. Six words out of twelve cuts the guessing problem down enormously. Never share any part of it.



Accounts

No notes. Let the slide speak.



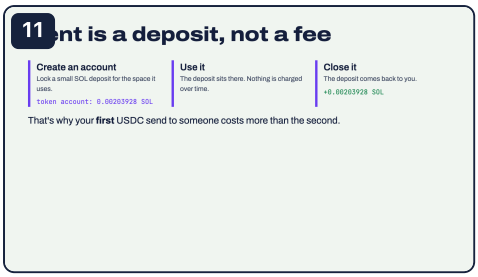
One shape for everything

Click through the four tabs. Same fields every time; only the values differ.

- lamports: the SOL balance. 1 SOL = 1,000,000,000 lamports.
- data: raw bytes. Only meaningful to the program that owns the account.
- owner: the program allowed to change this account's data and take its lamports. This is the single most important field for security. A token account owned by anything other than the Token Program is fake.
- executable: true means the account is a program.

Key idea for builders and analysts: programs don't store data inside themselves. They store it in separate accounts they own. A swap pool is an account owned by the DEX program. Your USDC balance is an account owned by the Token Program.

The numbers in the deposit rows are the real rent-exempt minimums: 1,461,600 lamports for an 82-byte mint, 2,039,280 lamports for a 165-byte token account.

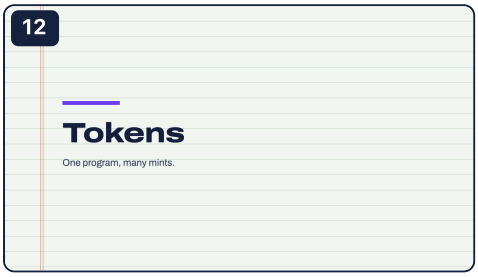


Rent is a deposit, not a fee

The name "rent" is historical. Today every account must hold a minimum balance proportional to its size ("rent-exempt"), and that balance comes back when the account is closed.

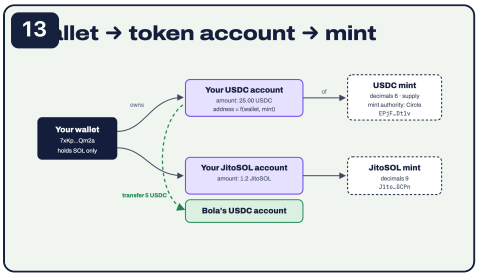
The first time you send USDC to someone who has never held it, your transaction creates their USDC account, so you pay its deposit (about 0.002 SOL). Second time, the account exists, so you pay only the tiny fee.

Wallets like Phantom let you close empty token accounts and reclaim the SOL. Some "claim your free SOL" sites do exactly this, legitimately or as bait. Check what a site asks you to sign.



Tokens

No notes. Let the slide speak.

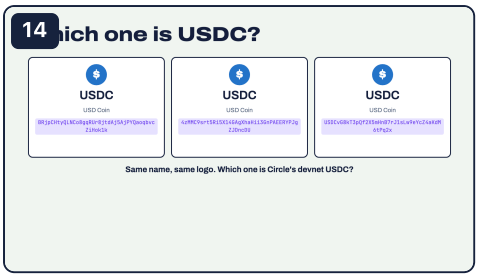


Wallet → token account → mint

The diagram that weeks 3 and 4 depend on. Draw it on a whiteboard too.

1. Your wallet: a System Program account holding SOL.
2. The USDC mint: one account that defines USDC. Decimals (6), total supply, and who is allowed to mint more (Circle) or freeze accounts.
3. Your USDC token account: holds your USDC balance. Its address is derived from your wallet address plus the mint, so anyone can work out where your USDC lives. That's the Associated Token Account, or ATA.
4. Another token, another token account. One per token you hold.
5. Sending to Bola moves USDC from your token account to Bola's token account. Your wallet signs, but the balance change happens in the token accounts.

Ethereum comparison: an ERC-20 is a contract with a table of balances inside it. On Solana, the code (Token Program) is shared by every token, the definition is the mint account, and each holder's balance is its own account.

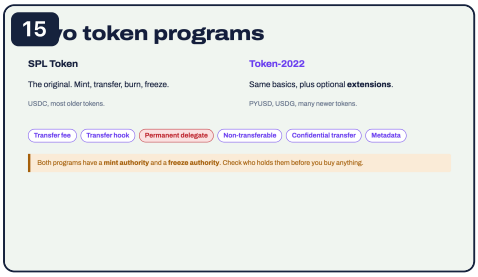


Which one is USDC?

Teaching moment straight from the handbook. Orca's devnet examples use their own devUSDC mint (BRjpCH...k1k). Circle's devnet USDC from faucet.circle.com is a different mint (4zMMC9...cDU). Both are "USDC" on devnet. You cannot swap one for the other 1:1, because they are different tokens.

The third card is made up for this slide, which is the point: anyone can create a token with any name and logo. Scam tokens copy names and logos all the time.

Rule: the mint address is the identity. Check it against the issuer's own documentation. On mainnet, Circle's USDC is EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v.

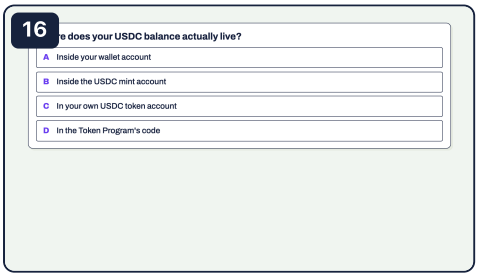


Two token programs

Extensions, in plain words:

- Transfer fee: the token takes a cut of every transfer.
- Transfer hook: every transfer also calls another program, which can block or log it.
- Permanent delegate: one address can move or burn anyone's balance of this token, forever. Useful for regulated assets (a court order), dangerous in a random memecoin. Red flag unless you know why it's there.
- Non-transferable: soulbound, can't be sent.
- Confidential transfer: hides amounts.
- Metadata: name, symbol and logo stored on the mint.

Mint authority: can create more tokens (dilution). Freeze authority: can freeze any holder's account. For USDC, Circle holds both, which is expected for a regulated stablecoin. For a memecoin, either one is a warning. Week 5 turns this into a checklist.



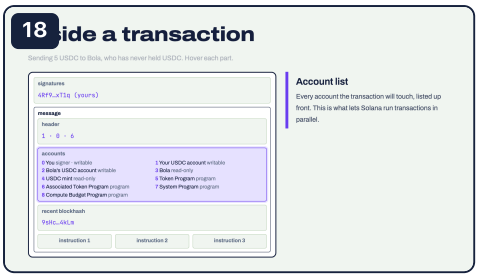
Quiz: Where does your USDC balance actually live?

No notes. Let the slide speak.



Transactions

No notes. Let the slide speak.



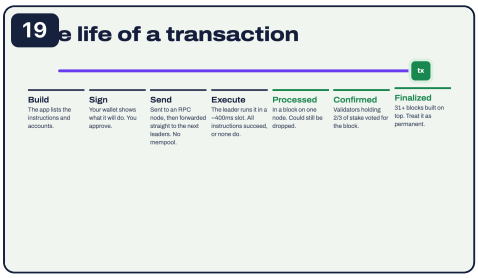
Inside a transaction

A transaction is a list of instructions plus the signatures that authorise them.

Walk through it top to bottom. The things to remember:

- The first signature is the transaction's ID.
- Every account is listed up front, with flags: does it sign, is it written to. This is what lets the runtime run unrelated transactions in parallel (week 1).
- The recent blockhash expires after about 150 blocks, roughly a minute. That's why a transaction you left in a wallet popup for five minutes fails.
- Instructions run in order, and all succeed or the whole transaction fails. That all-or-nothing property is called atomic.

A transaction can be at most 1,232 bytes. That limit is why complex DeFi transactions sometimes use "address lookup tables" to compress the account list.



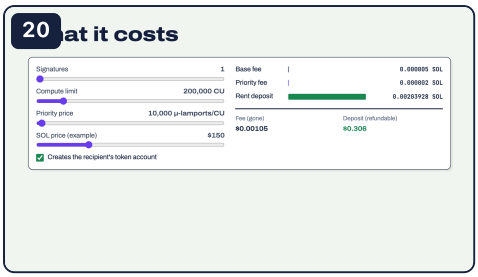
The life of a transaction

One stage per click.

Build: the app (say, a DEX front end) decides what instructions to send.
Sign: your wallet shows a preview. This is your last chance to catch a malicious request.
Send: the signed transaction goes to an RPC node, which forwards it directly to the validators scheduled to be leader soon. Solana doesn't use a shared mempool the way Ethereum does.
Execute: the leader runs it in its slot (about 400ms).
Processed, confirmed, finalized are the three "commitment levels" you'll see in explorers and APIs.

- Processed: in a block on the node you asked. Can still be dropped.
- Confirmed: validators with two thirds of the total stake voted for that block. Wallets usually show success here.
- Finalized: at least 31 more blocks have been built on top. For accounting purposes, treat this as settled.

Alpenglow, a new consensus protocol aimed at much faster finality, was approved by validator vote on 2 Sep 2025 (SIMD-0326, 98% yes). It went live on testnet (24 Sep 2026) and devnet (25 Sep 2026), and mainnet was scheduled for about 21 Oct 2026 (epoch 1064). Check solana.com/upgrades/alpenglow before quoting finality times: after it ships, the commitment levels on this slide change.



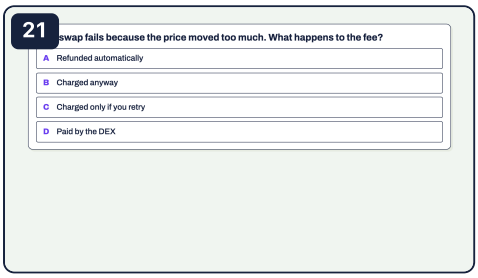
What it costs

Drag the sliders.

Base fee: 5,000 lamports per signature. Half of it is burned, half goes to the validator.
Priority fee: optional. Compute limit times the price you set per compute unit (in micro-lamports). All of it goes to the validator. Used when the network or a hot account is busy.
Rent deposit: only if the transaction creates an account, and you get it back when the account closes.

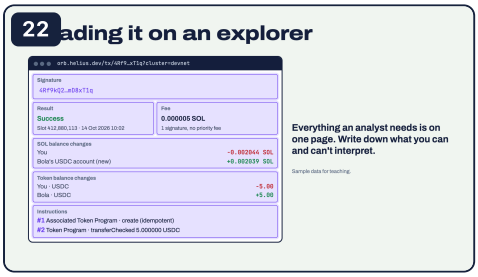
Finance connection: at these costs, a $5 trade is worth doing. On Ethereum mainnet in busy periods, gas could be several dollars, which killed small trades entirely. Fees shape who can use DeFi.

Failed transactions still pay the fee. The validator did the work of running it.



Quiz: Your swap fails because the price moved too much. What happens to the fee?

No notes. Let the slide speak.



Reading it on an explorer

This is the task in miniature. Click through each part of the page.

Point out the SOL balance changes: you paid 0.000005 SOL in fees and 0.00203928 SOL as the deposit for Bola's new USDC account. Total 0.00204428, shown rounded.

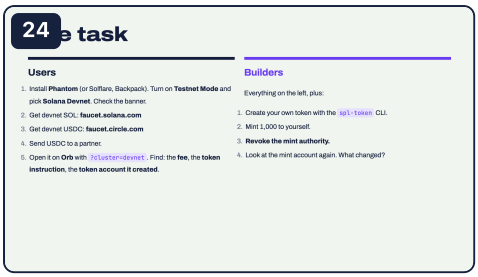
The analyst habit (from the curriculum): separate what you observed from what you interpreted. "The transaction shows a transferChecked of 5 USDC" is observed. "Ada paid Bola for lunch" is interpretation.

Orb's "Explain with AI" button is useful for a first read. Always check it against the raw instructions.



This week's task

No notes. Let the slide speak.



The task

Phantom: Settings → Developer Settings → Testnet Mode, then choose Solana Devnet. A banner shows when testnet mode is on.

Circle's devnet USDC mint: 4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU. Have mentees confirm the mint in Orb.

If the SOL faucet rate-limits the room (it limits by IP), send each mentee 1 devnet SOL from the teacher wallet you funded before class.

Pair people up for step 4 so everybody both sends and receives.



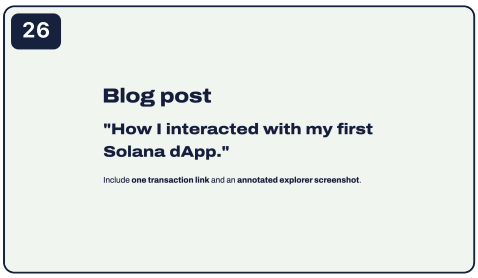
Builders: your own token

Step through with clicks.

After revoking the mint authority, nobody can ever mint more of this token. That's a one-way door. Revoking is what serious token launches do to prove a fixed supply, and it's the first thing to check on any token you're thinking of buying (week 5).

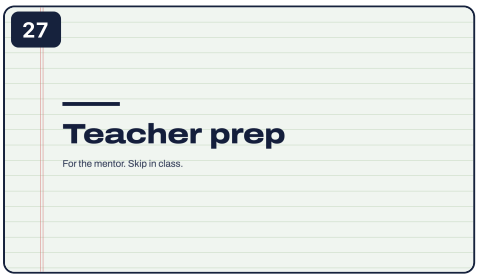
If `solana airdrop` is rate-limited, use faucet.solana.com in a browser with the address from `solana address`.

Optional extension: create a Token-2022 token with `spl-token create-token --program-2022` and add an extension like a transfer fee.



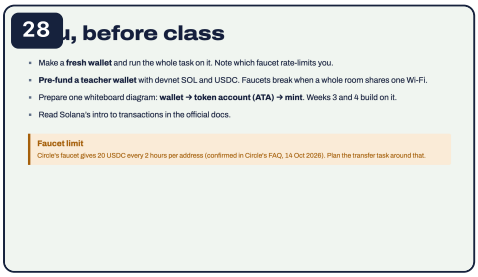
Blog post

No notes. Let the slide speak.



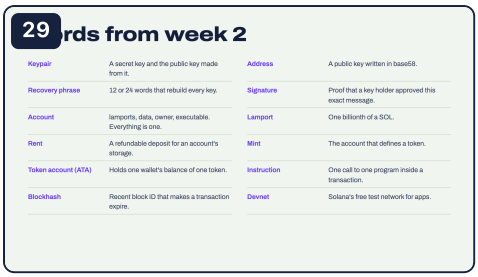
Teacher prep

No notes. Let the slide speak.



You, before class

No notes. Let the slide speak.



Words from week 2

No notes. Let the slide speak.